Anexo V. El Scanner y close()

Cómo abrir y cerrar correctamente la lectura del teclado

1. Recordatorio: ¿qué es un Scanner?

Cuando queremos que el usuario escriba algo con el teclado (su nombre, un número…), necesitamos un **Scanner**. Podemos imaginarlo como una **puerta** entre el teclado y nuestro programa: todo lo que el usuario escribe entra en el programa a través de esa puerta.

Código

```
Scanner teclado = new Scanner(System.in);
```

Trozo de código	Qué significa
`Scanner teclado`	Creamos una variable llamada *teclado* que guardará un Scanner.
`new Scanner(...)`	Fabricamos un Scanner nuevo.
`System.in`	El teclado del ordenador (*in* viene de *input*, entrada).

2. ¿Qué es close()?

Close significa **cerrar** en inglés. La instrucción `close()` sirve para **cerrar la puerta** cuando ya no vamos a leer nada más del teclado. Se escribe así:

Código

```
teclado.close();
```

Se pone **al final del programa**, cuando ya hemos leído todo lo que necesitábamos. Por ejemplo:

Código

```
import java.util.Scanner;

public class Saludo {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);   // 1. Abrimos la puerta

        System.out.print("Introduce tu nombre: ");
        String nombre = teclado.nextLine();         // 2. Leemos

        System.out.println("¡Hola, " + nombre + "!");

        teclado.close();                            // 3. Cerramos la puerta
    }
}
```

3. ¿Es obligatorio cerrarlo?

En los programas pequeños que hacemos ahora, **no pasa nada grave si se nos olvida**: cuando el programa termina, el ordenador cierra solo todo lo que había quedado abierto. Aun así, **lo vamos a cerrar siempre**, por dos razones:

- **Es una buena costumbre.** Más adelante usaréis el Scanner para leer ficheros, y ahí olvidar cerrarlo sí puede dar problemas.
- **Es lo que espera cualquier programador** que lea tu código.

> **Ojo con IntelliJ IDEA**
> IntelliJ **no te avisa** si se te olvida cerrar el Scanner: el programa se ejecuta igual y no aparece ningún mensaje. Por eso tienes que acordarte tú de escribir `teclado.close();` al final.

4. Lo más importante: el teclado solo hay uno

Cuando cierras un Scanner que lee del teclado, **no solo cierras el Scanner: cierras también el teclado para todo el resto del programa**. Y una vez cerrado, ya no se puede volver a abrir.

> **Piénsalo así**
> Imagina una habitación con una sola puerta. Puedes poner a dos personas vigilándola (dos Scanner), pero la puerta sigue siendo una. Si una de ellas la cierra con llave y se lleva la llave, nadie puede volver a usarla, por mucho que quiera.

Por no saber esto aparecen los dos errores más típicos con el Scanner. Vamos a verlos.

5. Error típico 1: cerrar el Scanner demasiado pronto

Este programa pide el nombre y la edad, pero cierra el Scanner **antes** de haber terminado de leer:

Código con error

```
import java.util.Scanner;

public class CerrarPronto {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Nombre: ");
        String nombre = teclado.nextLine();
        teclado.close();                    // ¡Demasiado pronto!

        System.out.print("Edad: ");
        int edad = teclado.nextInt();       // Aquí falla

        System.out.println(nombre + " tiene " + edad + " años.");
    }
}
```

Lo que se ve en pantalla (en negrita, lo que escribe el usuario)

```
Nombre: Ana
Edad: Exception in thread "main" java.lang.IllegalStateException: Scanner closed
```

¿Qué ha pasado?

Después de `teclado.close()` el Scanner ya está cerrado. Cuando intentamos leer la edad, Java se queja: `Scanner closed` significa *"el Scanner está cerrado"*. El programa ni siquiera te deja escribir la edad.

Solución: el `close()` va siempre al final, después de la última lectura.

6. Error típico 2: usar dos Scanner

Aquí alguien ha pensado: *"Si cierro uno, uso el otro"*. Parece buena idea, pero no funciona:

Código con error

```
import java.util.Scanner;

public class DosScanner {
    public static void main(String[] args) {
        Scanner teclado1 = new Scanner(System.in);
        Scanner teclado2 = new Scanner(System.in);

        System.out.print("Nombre: ");
        String nombre = teclado1.nextLine();
        teclado1.close();                    // ¡Cierra el teclado para todos!

        System.out.print("Edad: ");
        int edad = teclado2.nextInt();       // Aquí falla

        System.out.println(nombre + " tiene " + edad + " años.");
    }
}
```

Lo que se ve en pantalla (en negrita, lo que escribe el usuario)

```
Nombre: Ana
Edad: Exception in thread "main" java.util.NoSuchElementException
```

¿Qué ha pasado?

Al hacer `teclado1.close()` no solo se cierra `teclado1`: se cierra el teclado. `teclado2` sigue abierto, pero ya no tiene de dónde leer, así que el programa se detiene con un error.

`NoSuchElementException` significa, más o menos, *"no hay nada que leer"*.

7. La forma correcta

Un solo Scanner, que se crea al principio y se cierra **una sola vez, al final**, cuando ya se ha leído todo:

Código correcto

```
import java.util.Scanner;

public class UnScanner {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);  // Un solo Scanner

        System.out.print("Nombre: ");
        String nombre = teclado.nextLine();

        System.out.print("Edad: ");
        int edad = teclado.nextInt();       // Usamos el mismo Scanner

        System.out.println(nombre + " tiene " + edad + " años.");

        teclado.close();                    // Se cierra al final
    }
}
```

Lo que se ve en pantalla (en negrita, lo que escribe el usuario)

```
Nombre: Ana
Edad: 20
Ana tiene 20 años.
```

Fíjate en que no hace falta un Scanner para cada dato: **el mismo Scanner sirve para leer todo lo que necesites**, ya sean textos o números.

8. Resumen

Las tres reglas del Scanner

1. **Un solo Scanner** para el teclado en todo el programa.
2. **Ciérralo una sola vez, al final**, con `teclado.close();`
3. **Después de cerrarlo, ya no se puede leer más** del teclado.